

MEDIATEK

MT8676 Android Secure Boot User Manual

Version: 1.0
Release date: 2024-09-19

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-09-19	Heaven Zhang	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

List of Tables 3

1 Secure Boot 4

1.1 Overview 4

1.1.1 Abbreviations 4

1.1.2 BootRom 4

1.2 Secure Boot 5

1.2.1 Secure Boot Check Flow 5

1.3 Download DAA (DAA) 6

1.4 Security Feature Configuration 7

1.4.1 Generate A Key Pair 7

1.4.2 Enable Secure Boot 8

1.4.3 Compile Software 9

1.4.4 Sign the Preloader 9

1.4.5 Sign Other Images 9

1.5 Sign the DA 10

1.6 Configure efuse.xml 11

1.7 Generate Authfile 11

Exhibit 1 Terms and Conditions 12

List of Figures

Figure 1-1. Secure boot check flow 6

Figure 1-2. dakey.h 8

Figure 1-3. DA keys under resignda 10

List of Tables

Table 1-1. Abbreviations 4

1 Secure Boot

1.1 Overview

This document aims to provide an overview of the security boot feature in the MT8676 SoC and its accompanying SDK. During the product's design and deployment cycle, functionalities and features may change, which will be recorded in subsequent versions of this document. Additionally, how to enable security features is also explained in this document.

1.1.1 Abbreviations

Table 1-1. Abbreviations

Abbreviation	Explanation
DA	Download Agent (including DA_BR and DA_PL)
DAA	Download Agent Authentication
DRAM	Data Random Access Memory
eFuse	Electronic Fuse
eMMC	Embedded MultiMedia Card
EVB	Evaluation Board
NVM	Non-Volatile Memory
ROM	Read-only Memory
SBC	Secure Boot Check
SDK	Software Development Kit
SRAM	Static Random Access Memory
TEE	Trusted Execution Environment
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
USBDL	USB Download

1.1.2 BootRom

BootROM (BROM) is a software inside the System on Chip (SoC) ROM that cannot be modified. It is the first software executed by the application processor. The main tasks are summarized as follows.

- Basic hardware configuration of the SoC, such as setting up PLL/clock, is performed to enable startup.
- To boot the system, the first stage bootloader is loaded from the boot device (UFS/eMMC/NAND/...).
- Communication is established with the host PC to load Download Agent (DA) software for mirroring downloads of external FLASH that lacks software or has software corruption.
- Multiple secure boot verification keys should be used to conduct secure boot checks on the first-stage bootloader or download agent.

1.2 Secure Boot

1.2.1 Secure Boot Check Flow

The secure boot check flow is used to establish a trusted execution chain from the hardware root of trust. It can be enabled by burning SBC_EN in the eFuse. This process is executed every time the system recovers from a power-on reset.

Figure 1-1 illustrates the basic secure boot check flow. A detailed description is as follows.

1. After power-on reset, BROM uses the following process to verify the Secure Boot Check (SBC) public key (SBC_PUBK) in non-volatile memory (NVM), such as eMMC or NAND. BROM sequentially uses the public key hash (SBC_PUBK_HASH) from eFuse.
 - 1) Read the hash value (SBC_PUBK_HASH) of the secure boot check public key (SBC_PUBK) from eFuse
 - 2) Read the SBC public key (SBC_PUBK) from NVM.
 - 3) Calculate the hash value of the data from (2).
 - 4) Check if the data from (1) and (3) are the same.
2. BROM loads and verifies the Preloader, which is the first stage bootloader. It first reads the Preloader from NVM to SoC's SRAM and authenticates the Preloader using Secure Boot Check Public Key (SBC_PUBK). If the Preloader verification is successful, the Preloader will be executed. The verification method uses SHA256 to calculate a hash value, RSA (2048 bits) and MTK or PSS padding (determined by the header of the first stage bootloader) for verification.

Note:

 - If the external storage is NAND flash, BROM supports the second copy of Preloader. When BROM fails to load the first copy of Preloader, it will attempt to load/authenticate the second copy of Preloader.
3. After the first-stage bootloader, a certificate chain is introduced for the verification of the second-stage bootloader, Linux kernel, and other images. The certificate chain is designed for key management delegation, with each image being able to use a different pair of keys. Each image is accompanied by two certificates, CERT1 and CERT2, both of which are in x509v3 format. This involves two pairs of keys, the root key pair and the image key pair. CERT1 is signed by the root private key, while CERT2 is signed by the image private key. Therefore, the verification step would be, after CERT1 is verified with the public key (SBC_PUBK), the image public key extracted from CERT1 is used to verify CERT2. It is important to note that the CERT1 of each image is signed by the same root private key, but the CERT2 of each image may or may not be signed by the same image private key.
4. Preloader loads the second stage bootloader to the non-secure DRAM area from NVM and verifies it. It uses the public key embedded in Preloader to authenticate CERT1 of the second stage bootloader and uses the image public key extracted from CERT1 to authenticate CERT2 of the second stage bootloader. If the verification of the second stage bootloader is successful, it will be executed after TEE initialization. The verification method uses SHA256 to calculate hash value, RSA (2048-bit) and MTK or PSS padding for verification.
5. Preloader also loads TEE to a secure DRAM area from NVM and verifies it. It uses the public key inside the first stage bootloader to authenticate CERT1 of the TEE image, and uses the image public key extracted from CERT1 to authenticate CERT2 of the TEE image. If TEE verification is successful, TEE will be executed, and then the system jumps to the second stage bootloader. The verification method uses SHA256 to calculate hash value, RSA (2048 bits) and MTK or PSS padding for verification.

6. The second stage bootloader also uses embedded public keys and image public keys to verify CERT1 and CERT2 of the Linux kernel and other images that need to be verified at this stage. The verification method uses SHA256 to calculate hash value, RSA (2048 bits) and MTK or PSS padding for verification.
7. After executing on a Linux system, it is possible to verify read-only images using dm-verity in the Linux kernel (on block devices).

Note:

- Currently, dm-verity only supports block devices in Linux (EXT4 on eMMC).

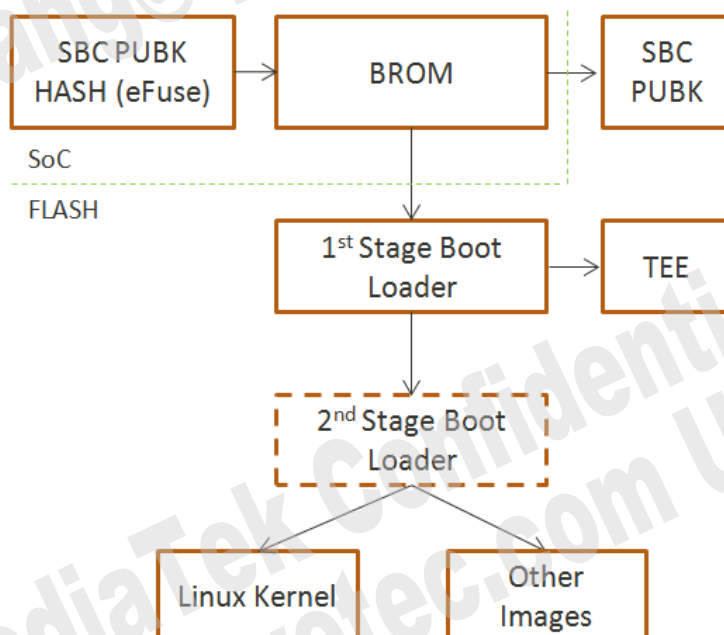


Figure 1-1. Secure boot check flow

1.3 Download DAA (DAA)

When the software in the external storage (eMMC/NAND/...) is empty or damaged, the host PC tool can communicate with the SoC's BROM, load DA into the SoC's SRAM, and execute DA to initiate the image downloading process. When Enable_DAA is burned in, DA will be authenticated by BROM, and this process is referred to as DA Authentication (DAA).

1. The host PC tool sends the authentication file (AuthFile) to BROM via USB or UART
2. BROM authenticates AuthFile.
 - 1) BROM reads the Secure Boot Check (SBC) public key (SBC_PUBK) from AuthFile, and uses the public key hashes (SBC_PUBK_HASH~SBC_PUBK_HASH1) and corresponding disable bits (SBC_PUBK_HASH_DIS~SBC_PUBK_HASH1_DIS) to authenticate the hash of the SBC key.
 - 2) BROM uses SBC_PUBK to verify AuthFile.
 - 3) BROM obtains the DAA key from the AuthFile.
3. The host PC tool sends the DA.
4. BROM uses the DAA key to verify DA through SHA256/RSA-2048.
5. BROM jumps to DA to execute the firmware download process.

Note:

- SBC_PUBK and DAA keys can be different to increase the security level.
- In addition to BROM, the first-stage bootloader also supports the DA protocol, which can be used to verify DA using SHA256/RSA2048.

1.4 Security Feature Configuration

1.4.1 Generate A Key Pair

A pair of root keys (comprising private and public keys) should be generated. Utilize `pem_to_der.py` to convert the key format to DER format in `vendor/MediaTek/proprietary/scripts/sign-image_v2/der_extractor/`.

1. Command to generate private key:

```
openssl genrsa -out root_prvk.pem 2048
python pem_to_der.py root_prvk.pem root_prvk.der
```

2. Command to generate public key:

```
openssl rsa -in root_prvk.pem -pubout > root_pubk.pem
python pem_to_der.py root_pubk.pem root_pubk.der
```

The image key pair (`img_prvk.pem/img_pubk.pem`) for signing and verifying images and the DA key pair (`da_prvk.pem/da_pubk.pem`) for signing and verifying DAs are generated using the same method.

1.4.1.1 Create oemkey.h

Export the root key (`root_pubk.der`), and generate `oemkey.h` using `der_extractor`. This tool is located in `vendor/MediaTek/proprietary/scripts/sign-image_v2/der_extractor/` directory. Please place `oemkey.h` at the following path:

[DA] `$DA_Kit/Raphael-da/custom/$PLATFORM/oemkey.h`

[PL] `$PL/custom/$PROJECT/inc/oemkey.h`

[LK] `$LK/target/$PROJECT/inc/oemkey.h`

Command:

```
chmod 777 der_extractor
./der_extractor root_pubk.der oemkey.h ANDROID_SBC
```

1.4.1.2 Create dakey.h.

The `dakey.h` file contains the `DA_PL` public key used by the preloader to verify `DA_PL.bin`. Subsequently, `DA_PL.bin` is signed using the corresponding private key.

Command:

```
chmod 777 der_extractor
./der_extractor da_pubk.der dakey.h ANDROID_SBC
```

Place dakey.h in the following path:

[PL] \$PL/custom/\$PROJECT/inc/dakey.h

Note:

- After generating dakey.h, please replace all "OEM" with "DA".

```
#ifndef _DAKEY_
#define _DAKEY_
#define DA_PUBK_SZ 256
#define DA_PUBK {0xD1, 0x64, 0x03, 0x46, 0x6C, 0x
0xE3, 0x32, 0xE1, 0x7D, 0xC0, 0
0x4E, 0xAA, 0xC2, 0xCB, 0x30, 0
0x68, 0x20, 0x0D, 0xE1, 0x7D, 0
0x73, 0x70, 0x50, 0xF0, 0x84, 0
0x0B, 0xB0, 0xCD, 0x60, 0x68, 0
0x71, 0x5F, 0x6F, 0x6B, 0x57, 0
```

Figure 1-2. dakey.h

1.4.2 Enable Secure Boot

1.4.2.1 Configure the Preloader

- vendor/mediate/proprietary/bootable/bootloader/preloader/custom/<project>/<project.mk>
MTK_SECURITY_SW_SUPPORT=yes
- vendor/mediate/proprietary/bootable/bootloader/preloader/custom/<project>/<project.mk>
MTK_SEC_BOOT= ATTR_SBOOT_ONLY_ENABLE_ON_SCHIP
MTK_SEC_USBDL = ATTR_SUSBDL_ONLY_ENABLE_ON_SCHIP
ATTR_SBOOT_ONLY_ENABLE_ON_SCHIP means that secure boot and secure download are enabled according to efuse SBC field.

In the early stages of the project, eFuse operations may not be desired for the device. Secure boot and secure download can be enforced without using eFuse to initially verify the software's secure boot flow.

```
MTK_SEC_BOOT= ATTR_SBOOT_ENABLE
MTK_SEC_USBDL = ATTR_SUSBDL_ENABLE
```

1.4.2.2 Configure LK

```
vendor/mediate/proprietary/bootable/bootloader/lk2/project/<project.mk>
MTK_SECURITY_SW_SUPPORT=yes
```

1.4.2.3 Configure Kernel

32 bit kernel

```
<kernel path>/arch/arm/configs/<project>_debug_defconfig
<kernel path>/arch/arm/configs/<project>_defconfig
```

64 bit kernel

```
<kernel path>/arch/arm64/configs/<project>_debug_defconfig
<kernel path>/arch/arm64/configs/<project>_defconfig
```

```
CONFIG_MTK_SECURITY_SW_SUPPORT=y
```

1.4.3 Compile Software

Compile the entire project.

1.4.4 Sign the Preloader

1. Replace your `root_prvk.pem` at the following path:
`/vendor/mediatek/proprietary/bootable/bootloader/preloader/custom/${project}/security/chip_config/s/key/`
2. Clear compile preloader.

1.4.5 Sign Other Images

1.4.5.1 Generate the Keys for cert1 and cert2

1. Use `root_prvk.pem` and `img_prvk.pem` to generate `cert1` and `cert2_key`.
2. Run the following command.

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/img_key_deploy.py mt6897
cert1_key_path=${KEY_PATH}/root_prvk.pem cert2_key_path=${KEY_PATH}/img_prvk.pem
root_key_padding=pss 2>&1 | tee SecureGen.log
```

Note:

- Please do not enter the `./vendor/mediatek/proprietary/scripts/sign-image_v2/` directory to execute `img_key_deploy.py`. Please execute this command in the root directory of the code repository.
- Please check if `cert1` and `cert2_key` of all images have been updated at the following locations:
`vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert1\`
`vendor\mediatek\proprietary\custom\mt6897\security\cert_config\cert2_key\`

1.4.5.2 Sign Images

After `cert1` and `cert2_key` have been generated, a signing script can be executed to generate `<image>-verified.bin` or `<image>-verified.img`.

Command:

```
python ./vendor/mediatek/proprietary/scripts/sign-image_v2/SignFlow.py <platform> <project>
2>&1 | tee signflow.log
```

Or

```
./vendor/mediatek/proprietary/scripts/sign-image/sign_image.sh 2>&1 | tee signflow.log
```

The project must be lunched before compiling.

1.5 Sign the DA

Before signing the DA, ensure that `oemkey.h` has been configured according to [Section 1.4.1.1](#). Both the Boot Region of DA (DA_BR) and the Preloader of DA (DA_PL) utilize Python scripts for the signing process.

1. Key configuration (da_prvk.pem and epp_prvk.pem use RSA2048)

Place `da_prvk.pem` and `epp_prvk.pem` (Figure 1-3) at

`./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/`.

- DA_PL: `da_prvk.pem` should be paired with the public key set in `dakey.h`.
- DA_BR: The `da_prvk.pem` should be paired with the DAA public key in the `authfile`.

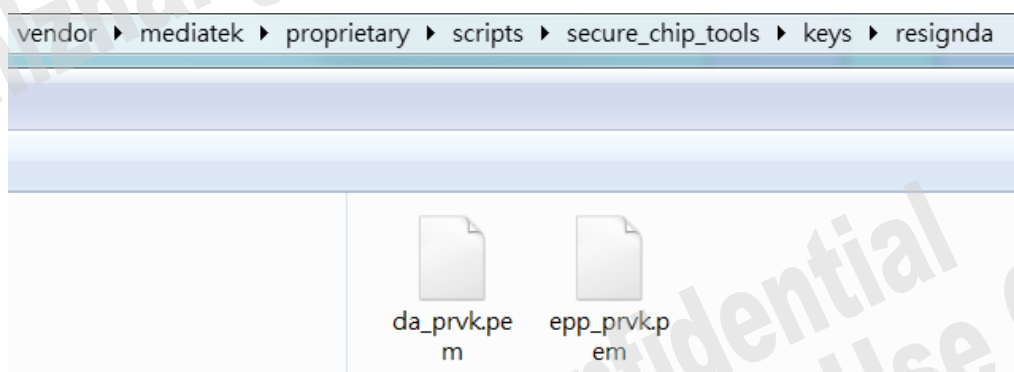


Figure 1-3. DA keys under resignda

Note:

- `epp_prvk` is equivalent to `da_prvk.pem`. Therefore, simply copy `da_prvk.pem` and rename it to `epp_prvk.pem`.
- DA_PL and DA_BR both use the PSS padding format, so if DA_PL and DA_BR are signed with the same key, the preloading program can use DA_BR. If DA_PL and DA_BR are signed with different keys, the preloading program can only use DA_PL.

2. Key path configuration

Usually the key path is preconfigured by default.

xref: `/vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/path_manager.py`

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download

```

14 *****
15
16 # Root Key For Preloader, x509 Img, RMA Key Cert, and Auth File Verification
17 ROOT_PRVK      = "custom_keys/root_prvk.pem"
18 ROOT_PUBK      = "custom_keys/root_pubk.pem"
19
20 # Img Key For Preloader And x509 Img Content Cert Verification
21 IMG_PRVK       = "custom_keys/img_prvk.pem"
22 IMG_PUBK       = "custom_keys/img_pubk.pem"
23
24 # Key For DA Verification
25 DA_PRVK        = "custom_keys/da_prvk.pem"
26 EPP_PRVK       = "custom_keys/epp_prvk.pem"
27

```

Set the key path

3. Place the DA that needs to be signed in `prebuilt/resignda/`, and execute the following command to sign the DA:

```
python MTK/resign_da.py prebuilt/resignda/DA_BR.bin MT6897
settings/Legacy/da/bbchips_pss.ini all out/resignda/DA_BR-resing.bin
```

After signing, the DA will be generated in the `out/resignda` directory.

1.6 Configure efuse.xml

Please apply for the efuse.xml file from MediaTek.

- In the efuse.xml, set the key type to pss and set pub-key-e to 010001. The pub-key-n field should be the same as the root public key in oemkey.h.

```
<sbk-pub-key>
  <key-type>pss</key-type>
  <pub-key-e>010001</pub-key-e>
  <pub-key-n>
    9BB734774443D77557A76E24B107
  </pub-key-n>
```

- Set SBC to true to enable the secure boot feature.

```
Enable_SBC="true"
```

- Set DAA to true to enable the DA verification function.

```
Enable_DAA="true"
```

1.7 Generate Authfile

The public key of DA is included in the authfile, which BROM uses to verify DA. Therefore, when DAA is enabled, authfile is required when downloading images using flashtool. All .ini files located at the following directory have been configured for this project.

```
./vendor/mediatek/proprietary/scripts/secure_chip_tools/settings/Legacy/authfile/
```

You should only replace the DA, epp, and .pem files of root private keys in the following directory.

```
./vendor/mediatek/proprietary/scripts/secure_chip_tools/custom_keys/
```

Authfile generation command:

```
python MTK/toolauth.py -i settings/Legacy/authfile/toolauth_key.ini -g
settings/Legacy/authfile/toolauth_gfh_config_pss.ini out/toolauth/auth_sv5.auth
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.